LAPORAN PRAKTIKUM
PEMROSESAN BAHASA ALAMI

Materi:
Pembobotan/TF-IDF

Disusun Oleh:
Firzal Akbar Ramadhan
2400018093

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
UNIVERSITAS AHMAD DAHLAN

2026

Pre Test

Bagian ini memuat hasil pengerjaan lembar jawaban Pre-Test pada praktikum pertemuan ini:

LEMBAR JAWABAN PRE-TEST PRAKTIKUM

Nama : Fraal Akbar R NIM : 24060180g3	Asisten: Paraf Asisten:	Tanggal: Nilai: 75

1) Term freq adalah jumlah setiap token/term pada dokumen

2) ~~Rumus Tf IDF = Tf x IDF~~
~~IDF~~

2) Rumus IDF = $\log_{10}\left(\frac{N}{DF}\right)$

DF = Dokumen frekuensi kemunculan token di setiap dokumen

N = Jumlah dokumen

$\log_{10}\left(\frac{3}{1}\right) = 0,477$

$\log_{10}\left(\frac{3}{2}\right) = 0,176$

$\log_{10}\left(\frac{3}{4}\right) = -0,124$

3) terversus

~~Dokumen Kebun TF~~

Token	D1	D2	D3	DF	IDF	TF IDF D1	D2	D3
kebun	1	0	1	2	0,176	0,176	0	0,176
binatang	1	1	2	4	-0,124	-0,124	-0,124	-248
itu	1	0	0	1	0,477			
terlihat	1	0	1	2	0,176			
sangat	1	0	0	1	0,477			
lucu	1	0	0	1	0,477			
berbagai	0	1	0	1	0,477			
macam	0	1	0	1	0,477			
terdapat	0	1	0	1	0,477			
dalamnya	0	1	1	2	0,176			
tersebut	0	0	1	1	0,477			
gemuk	0	0	1	1	0,477			
di	0	1	1	1	0,477			

Scanned with CamScanner

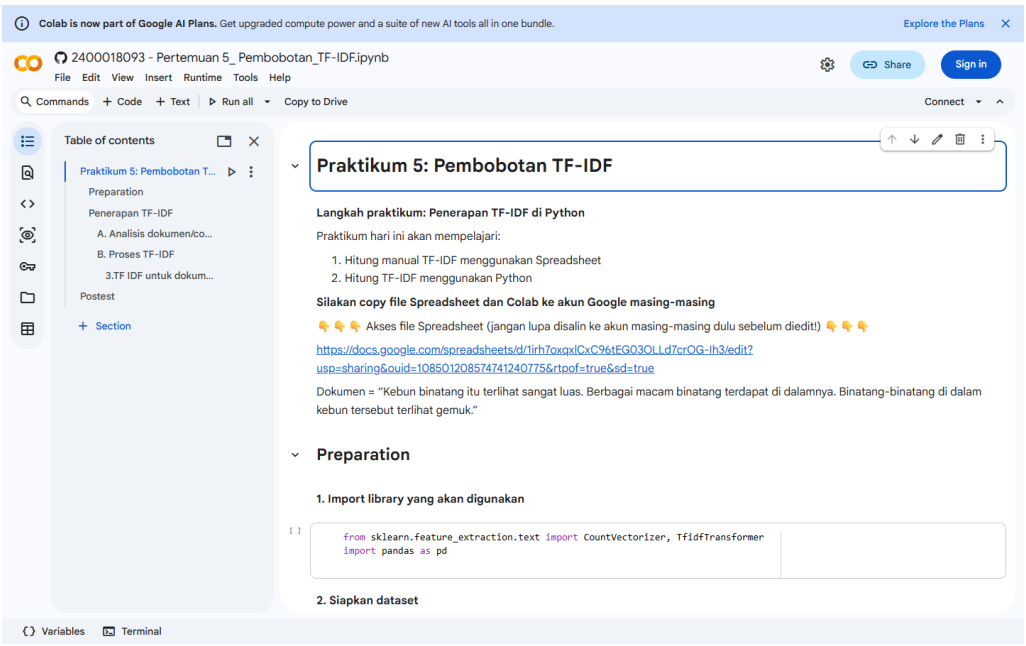
Gambar 5.1 Lembar Jawaban Pre-Test Praktikum Pertemuan 5

Hasil Praktikum

Praktikum pertemuan 5 membahas teknik representasi fitur numerik teks berbasis Pembobotan TF-IDF (Term Frequency - Inverse Document Frequency) untuk mengukur derajat kepentingan suatu kata dalam kumpulan dokumen.

5.1 Konsep Matematis TF dan IDF pada Google Colab

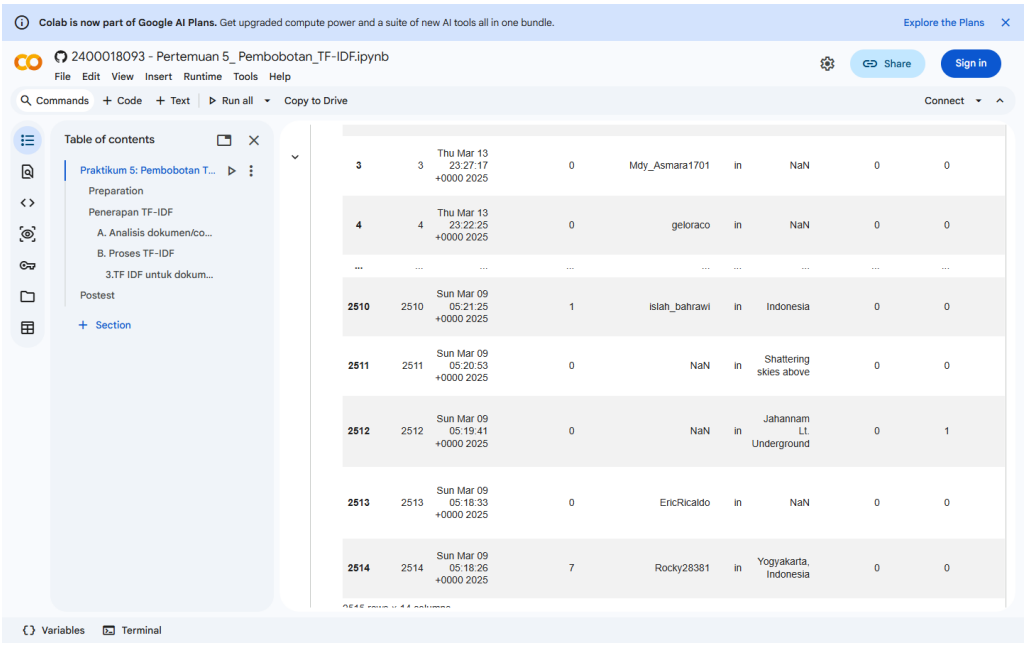
TF mengukur frekuensi kemunculan term dalam dokumen tertentu, sedangkan IDF mengukur kelangkaan kata di seluruh korpus dengan rumus IDF = log10(N/DF).



Gambar 5.2 Header Notebook Praktikum Pembobotan TF-IDF

5.2 Pembangunan Matriks Bobot TF-IDF dengan Scikit-Learn

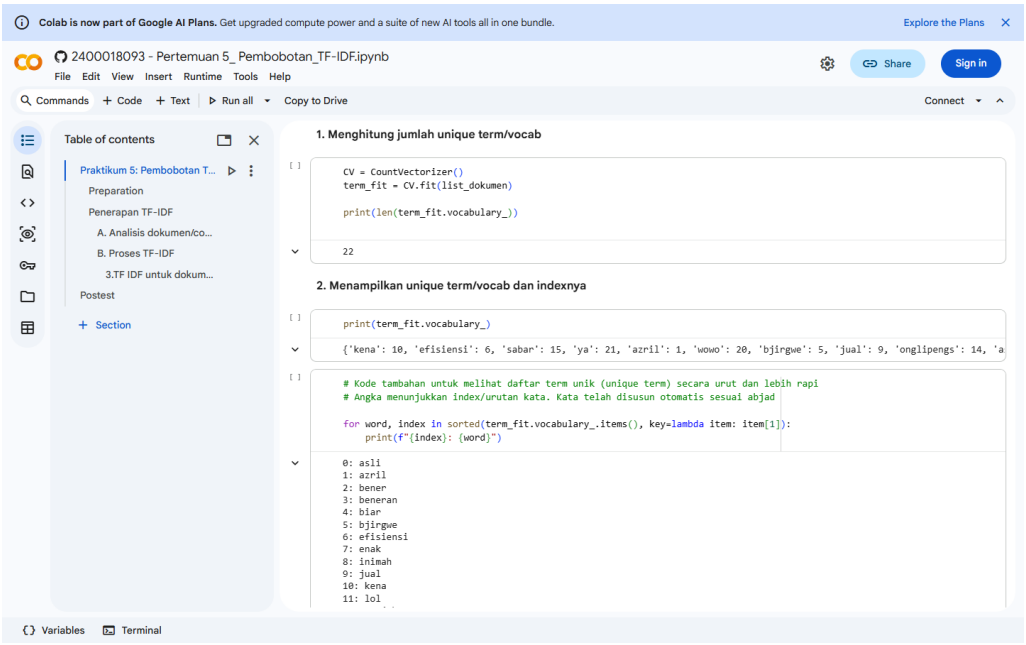
Pustaka TfidfVectorizer digunakan untuk mentransformasikan koleksi dokumen kalimat menjadi representasi matriks bobot numerik.



Gambar 5.3 Kode Perhitungan Bobot TF-IDF pada Korpus Dokumen

5.3 Analisis Matriks TF-IDF dan Kata Kunci Spesifik

Hasil bobot dianalisis: kata-kata yang umum muncul di semua dokumen mendapat bobot mendekati 0, sementara kata-kata pembeda yang unik mendapat nilai bobot tinggi.



Gambar 5.4 Matriks Hasil Bobot TF-IDF Antar Dokumen

Ulasan Praktikum

TF-IDF merupakan jembatan utama antara teks bahasa alami dengan algoritma pembelajaran mesin numerik. Teknik ini berhasil menyelesaikan kelemahan model Bag-of-Words biasa yang bias terhadap kata-kata yang terlalu sering muncul.

Post Test

Latihan post-test diambil langsung dari notebook '2400018093 - Pertemuan 5_ Pembobotan_TF-IDF.ipynb' dan file perhitungan Excel '2400018093 - Pertemuan 5 Hitung TF-IDF Manual.xlsx':

1. Pemilihan 5 Dokumen Kalimat Berbeda (Token < 10)

Dari dataset, diambil 5 kalimat pada indeks [31, 45, 89, 108, 132] kolom 'stemmed_text':
- D1: 'woahae nya kena efisiensi'
- D2: 'bmnya bgt efisiensi dongo'
- D3: 'deret program prioritas menteri pu kena efisiensi via'
- D4: 'ih gk had'
- D5: 'efisiensi'

```
subset2 = df.iloc[[31, 45, 89, 108, 132], [12]]
list_dokumen2 = [item[0] for item in subset2.values.tolist()]
print(list_dokumen2)
```

Kode 5.1 Ekstraksi 5 Dokumen Kalimat Subset Post-Test

2. Pembangunan Matriks Bobot TF-IDF dan Nilai IDF Kosakata

Pustaka CountVectorizer dan TfidfTransformer digunakan untuk memproses 5 dokumen tersebut. Dihasilkan 16 kosakata unik. Nilai IDF untuk 2 kosakata sampel:
- IDF('efisiensi'): 1.1823 (kata umum, muncul di 4 dokumen sehingga bobotnya rendah)
- IDF('dongo'): 2.0986 (kata spesifik, hanya muncul di 1 dokumen sehingga bobotnya tinggi)

Matriks bobot TF-IDF untuk seluruh 5 dokumen berhasil dikalkulasi secara presisi.

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfTransformer

CV = CountVectorizer()
term_fit2 = CV.fit(list_dokumen2)
komentar_term2 = term_fit2.transform(list_dokumen2)
tfidf_transformer2 = TfidfTransformer().fit(komentar_term2)
tfidf2 = tfidf_transformer2.transform(komentar_term2)

print('IDF efisiensi:', tfidf_transformer2.idf_[term_fit2.vocabulary_['efisiensi']])
print('IDF dongo:', tfidf_transformer2.idf_[term_fit2.vocabulary_['dongo']])
print('Matriks TF-IDF:\n', round(tfidf2, 3))
```

Kode 5.2 Perhitungan IDF dan Matriks TF-IDF dengan Scikit-Learn

3. Analisis Perbandingan: Hasil Perhitungan Manual Excel vs Coding Python

Apakah terdapat perbedaan hasil antara perhitungan manual rumus dengan pustaka Python?

Ya, terdapat perbedaan nilai numerik bobot akhir antara perhitungan lembar kerja manual

Excel dengan Scikit-Learn. Hal ini disebabkan oleh dua perbedaan formulasi matematis:

1. Formula IDF yang Digunakan:
   - Pada perhitungan manual standar: IDF = log10(N / DF).
   - Pada Scikit-Learn (smooth_idf=True): IDF = ln((1 + N) / (1 + DF)) + 1, menggunakan logaritma natural (basis e) dan menambahkan konstanta 1 pada pembilang dan penyebut guna menghindari pembagian dengan angka 0.
2. Normalisasi Vektor L2-Norm:
   - Perhitungan manual hanya mengalikan nilai TF mentah dengan IDF (TF * IDF).
   - Scikit-Learn menerapkan L2-Normalization (pembagian dengan akar kuadrat jumlah kuadrat elemen vektor baris) sehingga vektor bobot setiap dokumen memiliki panjang unit 1. Hal ini menjamin bahwa dokumen yang memiliki lebih banyak kata tidak secara otomatis memperoleh bobot yang lebih tinggi dibandingkan dokumen pendek.